Advanced Day 3 — The `this` Keyword

Student Document · Advanced JS · Session 3 of 15

★ ADVANCED JS — DAY 3 OF 15

60 min	30 min	15 min	15 min
TOTAL	TEACHING	HANDS-ON	Q&A

Today's Topics

#	Topic
1	What is `this` ?
2	The 4 Binding Rules
3	call / apply / bind
4	Arrow functions — Lexical `this`
5	`this` in classes
6	The "Lost this" Bug & Fixes
7	The Decision Tree

PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

Topic 1 **What is `this`?** Decided AT CALL TIME

`this` is a special keyword inside every function. It refers to "the object that called this function" — set **at call time**, NOT at definition time.

```
function whoAmI() {
  console.log(this);
}

whoAmI();                          // strict mode: undefined; non-strict:
window/global
```

```
const user = { name: "Priya", whoAmI };
user.whoAmI();                          // logs the user object — called as a method

const other = { name: "Aarav", whoAmI };
other.whoAmI();                         // logs other — same function, different this
```

Topic 2 The 4 Binding Rules

Default · Implicit · Explicit · new

#	Rule	Applies when	`this` becomes
1	Default	`fn()`	`undefined` (strict) / global
2	Implicit	`obj.fn()`	`obj`
3	Explicit	`fn.call(x)` / `.apply` / `.bind`	`x`
4	`new`	`new Fn()`	the brand-new object

```
// 1. Default — plain function call
function speak() { console.log(this); }
speak();                                // undefined (strict mode)

// 2. Implicit — method call
const car = {
  brand: "Tata",
  show() { console.log(this.brand); },
};
car.show();                             // "Tata"  ← this = car

// 3. Explicit — call/apply/bind
function intro(city) { console.log(`${this.name} from ${city}`); }
const u = { name: "Priya" };
intro.call(u, "Jaipur");                // "Priya from Jaipur"   ← this = u

// 4. new — constructor binding
function User(name) { this.name = name; }
const p = new User("Anaya");
console.log(p.name);                    // "Anaya"  ← this = the new object
```

FOLLOW ALONG

```
const obj = {
  n: 7,
  f() { return this.n; },
};
```

→

first?
second?

```
const g = obj.f;
console.log(obj.f());
console.log(g());
```

Topic 3 call / apply / bind

Explicit binding

Method	Invokes?	Args
`.call(this, a, b)`	Yes — now	Listed
`.apply(this, [a, b])`	Yes — now	Array
`.bind(this, a)`	No — returns new fn	Pre-set

```
function greet(city, lang) {
  console.log(`${this.name} from ${city} speaks ${lang}`);
}

const u = { name: "Priya" };

// call — invoke now, args listed
greet.call(u, "Jaipur", "Hindi");              // "Priya from Jaipur speaks Hindi"

// apply — invoke now, args as array
greet.apply(u, ["Jaipur", "Hindi"]);           // same output

// bind — returns a new function for later
const greetPriya = greet.bind(u, "Jaipur");    // partially applied: city pre-set
greetPriya("English");                          // "Priya from Jaipur speaks
English"
greetPriya("Marathi");                          // "Priya from Jaipur speaks
Marathi"

// Once bound, this CANNOT be re-bound
greetPriya.call({ name: "Aarav" }, "Tamil");   // still "Priya from Jaipur speaks
Tamil"
```

Topic 4 Arrow Functions — Lexical this

No own this

> Arrow functions have **NO own `this`**. `this` inside an arrow is whatever it is in the enclosing scope. `.call` / `.apply` / `.bind` can't change it.

```
const user = {
  name: "Priya",
```

```
  // Regular function — has its own this
  regular: function () {
    console.log(this.name);              // "Priya"  ← implicit binding
  },
  // Arrow — no own this; inherits from enclosing scope (here: module/global)
  arrow: () => {
    console.log(this.name);              // undefined ← arrow doesn't see user as this
  },
};

user.regular();   // "Priya"
user.arrow();     // undefined          ← surprise!  arrow as a method is usually
wrong

// Where arrow shines: nested callbacks
const team = {
  members: ["Priya", "Aarav", "Riya"],
  greetAll() {
    this.members.forEach((m) => {
      // Arrow here inherits 'this' from greetAll → which is 'team'
      console.log(`Hi ${m}, from team ${this.members.length}`);
    });
  },
};
team.greetAll();
// "Hi Priya, from team 3"
// "Hi Aarav, from team 3"
// "Hi Riya, from team 3"
```

⚠️ **Don't use arrows as object methods.** Inside an arrow on an object, `this` won't be the object. Use regular functions for methods. Use arrows for inner callbacks where you WANT to inherit `this`.

Topic 5 **`this` in Classes** Same as implicit

```
class User {
  constructor(name) {
    this.name = name;                    // this = the new instance
  }

  greet() {
    console.log(`Hi, I'm ${this.name}`); // this = whichever instance .greet() was
called on
  }
}

const a = new User("Priya");
const b = new User("Aarav");
```

```
a.greet();                          // "Hi, I'm Priya"
b.greet();                          // "Hi, I'm Aarav"

// Classic loss-of-this:
const greetFn = a.greet;            // pulled off as a plain function
// greetFn();                       // TypeError: cannot read 'name' of undefined
                                    //   (this is now undefined in strict mode)
```

Topic 6 The "Lost this" Bug & Fixes

Three fixes

```
class Counter {
  constructor() { this.count = 0; }
  inc() { this.count++; console.log(this.count); }
}

const c = new Counter();

// BUG — passed as plain function reference, this is lost
setTimeout(c.inc, 100);             // TypeError: cannot read 'count' of
undefined

// FIX 1 — bind
setTimeout(c.inc.bind(c), 100);     // works — this permanently bound to c

// FIX 2 — arrow wrapper (closes over c lexically)
setTimeout(() => c.inc(), 100);     // works — c.inc() called as a method

// FIX 3 — class field as arrow (modern syntax)
class CounterArrow {
  count = 0;
  inc = () => { this.count++; console.log(this.count); };  // arrow → lexical this
}
const ca = new CounterArrow();
setTimeout(ca.inc, 100);            // works — arrow's this is the instance
```

FOLLOW ALONG

```
const obj = {
  n: 10,
  m: () => console.log(this.n),
};
obj.m();
```

→ *prints?*

Topic 7 **The Decision Tree** Walk these in order

Step	Question	If yes → `this` is
1	Arrow function?	Outer scope's `this`
2	Called with `new` ?	The new object
3	Called with `.call` / `.apply` / `.bind` ?	The explicit value
4	Called as `obj.fn()` ?	`obj`
5	Plain `fn()` ?	`undefined` (strict) / global

PART 2 — HANDS-ON EXERCISE (15 MIN)

Task 1 Predict the `this`

- Type the following snippet exactly: a `user` object with `name: "Priya"` and a method `greet() { console.log(this.name); }` . Then call `user.greet()` . Then assign `const g = user.greet` and call `g()` .
- Predict the output of each call BEFORE running.
- Run. Note actual output.
- In a comment, explain why the second call lost the `this` .

Task 2 Fix it Three Ways

- Take this code: `class Timer { constructor() { this.sec = 0; } tick() { this.sec++; console.log(this.sec); } }`
- Create `const t = new Timer();` then `setInterval(t.tick, 1000);`
- Run it — observe the TypeError.
- Now fix it THREE different ways: (1) using `.bind` , (2) using an arrow wrapper, (3) using a class field arrow.
- Verify all three log 1, 2, 3, ...

Task 3 call / apply / bind

- Define `function describe(role, city) { console.log(`${this.name} is a ${role} from ${city}`); }`
- Create `const u = { name: "Aarav" }`
- Call `describe` with `u` as `this` using `.call` , then `.apply` , then create a bound version with `.bind` (pre-fill role to "developer") and call it.

- In a comment, write the one-line difference between the three.

Bonus **Arrow vs Regular Method**

- Build an object `team` with `members: ["Priya", "Aarav", "Riya"]` and TWO methods.
- Method A — `printRegular()` uses `forEach(function(m) { console.log(this.members.length, m); })`.
- Method B — `printArrow()` uses `forEach((m) => { console.log(this.members.length, m); })`.
- Call both. Note which one breaks and why.

PART 3 — VISUAL SUMMARY

The 4 Rules — Priority

Priority	Rule
Highest	Arrow → outer scope's `this`
	`new Fn()`
	`.call` / `.apply` / `.bind`
	`obj.fn()`
Lowest	Plain `fn()` → undefined / global

call / apply / bind

Method	When	Args format
`.call`	Now	Listed
`.apply`	Now	Array
`.bind`	Later	Pre-set

Lost-this Fixes

Fix	How
Bind	`obj.fn.bind(obj)`
Arrow wrapper	`() => obj.fn()`
Class field arrow	`fn = () => { ... }`

Quick Lookup

Code	`this`
`obj.method()`	`obj`
`const f = obj.m; f();`	`undefined` (strict)
`new Foo()`	new instance
`() => this`	outer `this`
`arr.forEach(fn)` (regular)	`undefined`

HOMEWORK

Practice tasks before next class · Advanced Day 3

1. Build an object `user = { name, greet }` where `greet` logs `this.name`. Now call `user.greet()`, `const fn = user.greet; fn()`, and `user.greet.call({ name: "X" })`. Predict each first, then run.
2. Take any callback bug you've hit before with a class method and fix it three ways (bind, arrow wrapper, class field arrow). Document each.
3. Write `function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }`. Use `.apply` to call it with an array `[1, 2, 3, 4, 5]`. Why does `.apply` shine here?
4. Take an arrow function `const f = () => console.log(this)` written at top-level of a module. Try to `.bind({ x: 1 })` it and call. What does it log? Why?
5. Read: `javascript.info/object-methods` (the "this" section) and `javascript.info/bind`.
6. *(Bonus)* Watch Akshay Saini's "Namaste JS — call, apply, bind". ~10 minutes.